FOR COMPLETE BEGINNERS

Half built. On purpose.

Complete Modern Java, episode 28: abstract classes, in Java and TypeScript

> A parent with work left unfinished: its finished methods call the unfinished ones, and each child fills them in.

Card adds a two percent fee; cash adds nothing. Both work out a total the same way: price plus fee.

Write the shared calculation once, in an abstract parent, and leave the fee to each payment type.

In this episode

- abstract class
- abstract method
- A finished method calling an unfinished one
- Two finished children
- Cannot create the parent
- Interface or abstract class
- TypeScript abstract at run time

Coming next

- Polymorphism (episode 29)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

PaymentMethod payment = new CardPayment(); totalCents(1000)	`1020`
a list of card and cash	`Card: 1020, Cash: 1000`
new PaymentMethod("Voucher")	`compile error`
pay.ts	`Card: 1020, Cash: 1000`
pay.ts with a PaymentMethod voucher	`TS2511; node: this.feeCents is not a function`

Every row was run, not predicted.

02 The abstract parent

totalCents is finished and calls feeCents, which is not. Abstract describes an incomplete class; not every method in it is abstract.

In our little shop each payment type decides its own fee; real payment systems have more rules. Fractions of a cent are dropped.

PaymentMethod.java

```
abstract class PaymentMethod {
    private String name;

    PaymentMethod(String name) {
        this.name = name;
    }

    String getName() {
        return name;
    }

    abstract int feeCents(int priceCents);

    int totalCents(int priceCents) {
        return priceCents + feeCents(priceCents);
    }
}
```

> A PaymentMethod variable holding a CardPayment printed 1020; the list printed Card: 1020 and Cash: 1000.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 The children fill the gap

These are ordinary, finished classes, so Java requires each one to fill in feeCents. You cannot create the parent itself.

CardPayment.java

```
class CardPayment extends PaymentMethod {
    CardPayment() {
        super("Card");
    }

    @Override
    int feeCents(int priceCents) {
        return priceCents * 2 / 100;
    }
}
```

> new PaymentMethod("Voucher"): PaymentMethod is abstract; cannot be instantiated.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 TypeScript abstract

tsc refuses to create the abstract class. node creates it anyway, and fails when the missing method is called.

pay.ts

```
abstract class PaymentMethod {
    abstract feeCents(priceCents: number): number;
    totalCents(priceCents: number): number {
        return priceCents + this.feeCents(priceCents);
    }
}
const voucher = new PaymentMethod("Voucher");   // TS2511
console.log(voucher.totalCents(1000));          // node: TypeError
```

> tsc: Cannot create an instance of an abstract class. node: this.feeCents is not a function.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Abstract classes, in one look

1. Shared work in the parent; the missing part left abstract.
2. Finished methods may call unfinished ones.
3. Every finished child fills in the gap.
4. Interface: a promise. Abstract class: partly built.
5. Challenge: VoucherPayment, 50 cent fee. Voucher: 1050.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.